

Artificial Intelligence - Lab 7

Constraint Satisfaction Problems (OR-Tools)

Name: Muhammad Rehan Siddiqui

Roll No: 24K-0707

Q1: Shift Assignment Problem

```
1  from ortools.sat.python import cp_model
2
3  def solve_shift_assignment():
4      model = cp_model.CpModel()
5
6      alice = model.NewIntVar(0, 2, 'Alice')
7      bob = model.NewIntVar(0, 2, 'Bob')
8      charlie = model.NewIntVar(0, 2, 'Charlie')
9
10     model.AddAllDifferent([alice, bob, charlie])
11     model.Add(alice != 0)
12
13     solver = cp_model.CpSolver()
14     status = solver.Solve(model)
15
16     if status == cp_model.OPTIMAL or status == cp_model.FEASIBLE:
17         shifts = {0: "Morning", 1: "Afternoon", 2: "Evening"}
18         print(f"Alice: {shifts[solver.Value(alice)]}")
19         print(f"Bob: {shifts[solver.Value(bob)]}")
20         print(f"Charlie: {shifts[solver.Value(charlie)]}")
21     else:
22         print("No solution found.")
23
24 print()
25 solve_shift_assignment()
26 print()
```

Q2: Cryptarithmic Puzzle (SEND + MORE = MONEY)

```
1 from ortools.sat.python import cp_model
2
3 def solve_cryptarithmic():
4     model = cp_model.CpModel()
5
6     s = model.NewIntVar(0, 9, 'S')
7     e = model.NewIntVar(0, 9, 'E')
8     n = model.NewIntVar(0, 9, 'N')
9     d = model.NewIntVar(0, 9, 'D')
10    m = model.NewIntVar(0, 9, 'M')
11    o = model.NewIntVar(0, 9, 'O')
12    r = model.NewIntVar(0, 9, 'R')
13    y = model.NewIntVar(0, 9, 'Y')
14
15    model.AddAllDifferent([s, e, n, d, m, o, r, y])
16    model.Add(s != 0)
17    model.Add(m != 0)
18
19    model.Add(
20        (s * 1000 + e * 100 + n * 10 + d) +
21        (m * 1000 + o * 100 + r * 10 + e) ==
22        (m * 10000 + o * 1000 + n * 100 + e * 10 + y)
23    )
24
25    solver = cp_model.CpSolver()
26    status = solver.Solve(model)
27
28    if status == cp_model.OPTIMAL or status == cp_model.FEASIBLE:
29        print(f"S: {solver.Value(s)} | E: {solver.Value(e)}")
30        print(f"N: {solver.Value(n)} | D: {solver.Value(d)}")
31        print(f"M: {solver.Value(m)} | O: {solver.Value(o)}")
32        print(f"R: {solver.Value(r)} | Y: {solver.Value(y)}\n")
33
34        send = solver.Value(s)*1000 + solver.Value(e)*100 + solver.Value(n)
35        *10 + solver.Value(d)
36        more = solver.Value(m)*1000 + solver.Value(o)*100 + solver.Value(r)
37        *10 + solver.Value(e)
38        money = solver.Value(m)*10000 + solver.Value(o)*1000 + solver.Value(
39            n)*100 + solver.Value(e)*10 + solver.Value(y)
40
41        print(f"Equation: {send} + {more} = {money}")
42    else:
43        print("No solution found.")
44
45    print()
46    solve_cryptarithmic()
47    print()
```

Q3: Factory Production Optimization

```
1 from ortools.sat.python import cp_model
2
3 def solve_factory_production():
4     model = cp_model.CpModel()
5
6     laptops = model.NewIntVar(10, 100, 'Laptops')
7     phones = model.NewIntVar(10, 100, 'Phones')
8     tablets = model.NewIntVar(10, 100, 'Tablets')
9
10    # Resource Constraints
11    model.Add(2 * laptops + 1 * phones + 1 * tablets <= 100)
12    model.Add(1 * laptops + 1 * phones + 1 * tablets <= 80)
13    model.Add(1 * laptops + 1 * phones + 2 * tablets <= 120)
14
15    # Objective: Maximize Profit
16    model.Maximize(500 * laptops + 300 * phones + 400 * tablets)
17
18    solver = cp_model.CpSolver()
19    status = solver.Solve(model)
20
21    if status == cp_model.OPTIMAL or status == cp_model.FEASIBLE:
22        print(f"Maximum Profit: ${solver.ObjectiveValue()}")
23        print(f"Laptops: {solver.Value(laptops)}")
24        print(f"Phones: {solver.Value(phones)}")
25        print(f"Tablets: {solver.Value(tablets)}")
26    else:
27        print("No solution found.")
28
29 print()
30 solve_factory_production()
31 print()
```